Listing 8-5. *Flicker from EEPROM Source Code*

```
#include <EEPROM.h>

boolean writeEEPROM = true;

int ledPin = 11;
byte flicker[] = {64, 22, 4, 28, 6, 130, 186, 120};
int i = 0;

void setup() {
  randomSeed(analogRead(A5));
  if (writeEEPROM == true)
    for (int i=0; i<(sizeof(flicker)/2); i++)
      EEPROM.write(i, flicker[i]);
  i = 0;
}

void loop() {
  analogWrite(ledPin, EEPROM.read(i));
  delay(random(100, 500));
  i++;
  if (i == (sizeof(flicker)/2)) i = 0;
}
```

To read and write data to the EEPROM, we need to use the EEPROM library. This is not included by default in our sketches so we have to use the following line to access the EEPROM functions:

```
#include <EEPROM.h>
```

We have also added the following line:

```
boolean writeEEPROM = true;
```

Since we only need to write the data once, this will allow us to upload and run the code once, then change this variable to `false`, re-upload the code, and this time it will only read the EEPROM data. That will keep us from using up our total read/write cycles too quickly. The values in the `flicker[]` array stay the same, but we changed the data type to `byte` because it is easiest if the information we want to store is already a `byte` data type. To store the data the first time, we use the following block of code:

```
if (writeEEPROM == true)
    for (int i=0; i<(sizeof(flicker)/2); i++)
      EEPROM.write(i, flicker[i]);
```

This loop, only activated when the variable `writeEEPROM` is set to true, increments through a `for` loop until the end of the array has been reached, in this case eight times, although we could easily add more elements. Each time through the loop we use the `EEPROM.write()` function that has the following syntax:

```
EEPROM.write(address, value)
```

The address can be a number from 0 to 1023 corresponding to the location of each byte in memory space, and the value is a number between 0–255. In our example, we start with the first EEPROM location, 0, and write the value of `flicker[]` at index 0. The loop increments each time, writing the next value in the array to the next space in memory until it has reached the end of the array.